

PROJECT TECHNICAL REPORT

Broker Feed Data Quality Monitor

An Automated DataOps Pipeline for Financial Trade Feed Validation

Prepared By Amegha V. S	Role Data Analyst
Project Category DataOps / Data Quality	Domain Financial Services
Tech Stack Python · MySQL · Power BI	Status Completed
Date May 2026	Version 1.0

1. Executive Summary

The Broker Feed Data Quality Monitor is an end-to-end automated DataOps pipeline designed to address a critical operational challenge in financial services: the reliable validation of incoming trade data feeds from multiple broker sources before they are loaded into production databases or downstream reporting systems.

In financial institutions, broker feeds arrive daily in the form of structured CSV files. These files frequently contain defects — missing trade prices, extreme price anomalies, and duplicate transaction records — caused by system transmission errors, schema mismatches, or upstream data quality failures. Without automated validation, these defects propagate silently into compliance reporting systems, causing audit failures, incorrect risk calculations, and hours of manual remediation.

This project simulates and solves exactly that problem. The pipeline ingests trade feeds from five simulated broker endpoints, executes three structured quality validation functions against every incoming record, classifies each identified defect by operational severity, persists structural audit trails to a relational MySQL database environment, and surfaces an enterprise-ready dashboard in Power BI. This deployment architecture enables data operations teams to effortlessly triage, investigate, and escalate feed violations before they propagate into downstream reporting environments.

Key Results at a Glance

5,010	164	3	5
Trade Records Processed	Total Issues Flagged	Issue Categories Detected	Broker Feeds Monitored

2. Business Problem Statement

2.1 The Real-World Challenge

Modern compliance platforms ingest millions of transaction logs daily from external broker endpoints. In practice, these incoming files routinely arrive corrupted due to a variety of root causes:

- Network transmission failures that truncate or corrupt field values during file transfer
- Schema mismatches when broker systems update their output formats without prior notice
- Upstream system errors that produce null values, negative prices, or impossibly large price figures
- Processing glitches that generate duplicate transaction records with identical Trade IDs but differing quantities

If a compliance platform blindly loads these unvalidated data streams into its primary production database, the consequences are severe:

- Critical system failures caused by NULL values in mandatory price fields that break downstream calculations
- Incorrect regulatory reporting due to inflated or missing position values
- Duplicate records that double-count trade volumes, distorting risk exposure metrics
- Audit vulnerabilities when data quality defects are only discovered during a regulatory review
- Hours of manual debugging for data operations teams who must trace issues back through raw files

2.2 The Solution Approach

This project engineers an automated, proactive DataOps pipeline that acts as a first line of defense between raw broker feeds and the production data environment. Rather than allowing defective data to enter the system, the pipeline intercepts, inspects, and quarantines each record

against structured quality constraints before any downstream loading occurs. Findings are persistently logged to a relational MySQL database table and seamlessly surfaced through an executive Power BI monitoring dashboard for comprehensive operational oversight.

3. Project Architecture & Technical Design

3.1 Enterprise Tech Stack

Layer	Technology	Purpose
Ingestion & Processing	Python 3 (Pandas, NumPy, OS)	Data generation, cleaning, anomaly detection, pipeline orchestration
Scripting Environment	Jupyter Notebook	Interactive development, step-by-step pipeline execution and output logging
Storage & Logging	MySQL Workbench	Relational schema for error log persistence; SQL analytical queries for insights
Visualisation & BI	Microsoft Power BI	Executive operational dashboard with KPI cards, broker health scorecards, issue breakdown
Data Format	CSV (Structured)	Broker trade feed simulation; outbound operational metric data for structured analytical dashboards and relational database storage
Version Control	OneDrive (Folder Structure)	Organised project structure: data_inputs, scripts, data_outputs, dashboard, reports

3.2 Project Folder Structure

The project follows a clean, professional directory structure that mirrors enterprise DataOps repository organisation:

File / Folder	Description
---------------	-------------

scripts/1_Data_Generation.ipynb	Automated data synthesis engine — generates 5 broker CSVs with injected errors
scripts/2_Data_Quality_Engine.ipynb	Algorithmic evaluation engine — runs 3 quality checks, produces error log and summary matrices
scripts/1_database_schema.sql	Production schema definition — creates broker_dataops_db and feed_errors table in MySQL
scripts/2_database_dump.sql	Relational data verification — verifies table row metrics and runs structured operational data queries
data_inputs/*.csv	5 simulated broker trade feed CSVs (Apex, Beacon, Summit, Vanguard, Horizon)
data_outputs/detailed_error_log.csv	Granular error log with Broker, TradeID, Issue Type, Severity for every flagged record
data_outputs/broker_error_summary.csv	Programmatic outbound metric log — aggregated error counts per broker for BI loading
data_outputs/broker_issue_matrix.csv	Structured matrix of flag types — cross-tabulation of broker source against validation error category
dashboard/DataOps_Feed_Monitor.pbix	Live Power BI executive interface with broker health scorecards and issue triage view
reports/Broker_Feed_Quality_Report.pdf	Comprehensive technical briefing document (this report)

3.3 Data Flow Architecture

The pipeline executes in four sequential layers:

- **Layer 1 — Data Ingestion:** Python reads all five raw broker CSV files from the data_inputs/ directory dynamically using pattern matching, capturing the specific source broker name from the input file properties.
- **Layer 2 — Quality Evaluation & Quarantine:** Each incoming record is evaluated across structural logic checks. Defective records are cleanly separated and written to a row-level audit trail file (detailed_error_log.csv) in data_outputs/ to isolate them before database loading occurs.
- **Layer 3 — Relational Logging:** The complete row-level error log dataset is ingested directly into the MySQL database environment (broker_dataops_db) into the transactional feed_errors schema for long-term historical tracking and auditing.

- **Layer 4 — Visual Reporting:** The executive Power BI monitoring application reads your structured analytical metrics straight from your folder outputs to power real-time dashboards, vendor health scores, and drill-down triage views.

4. Data Simulation & Generation (Notebook 1)

4.1 Synthetic Broker Feed Design

Because real broker trade data is proprietary and regulated, the project uses Python's random and datetime libraries to generate realistic, high-fidelity synthetic trade records that faithfully simulate the characteristics of live broker feeds. Five separate CSVs are generated, each representing a distinct broker endpoint.

4.2 Injected Error Scenarios

Each broker feed is programmatically seeded with the following defect types to create a realistic test environment:

- **Missing Prices (2% probability):** Randomly set the Price field to NaN/null, simulating a transmission failure or field truncation from the upstream broker system.
- **Price Anomalies (1% probability):** Randomly inject extreme values (-50.0 or 999,999.99) to simulate system overflow errors or corrupted numeric encoding — values no legitimate trade would carry.
- **Duplicate Trade IDs (50% broker probability):** For randomly selected brokers, five existing trade records are duplicated with modified quantities — simulating a double-submission event at the broker's trading gateway.

4.3 Generated Dataset Summary

Broker	Base Records	Final Records (with dupes)	Dupes Injected
Apex_Securities	1,000	1,005	5
Beacon_Trades	1,000	1,005	5
Summit_Clearing	1,000	1,000	0
Vanguard_Feeds	1,000	1,000	0
Horizon_Brokers	1,000	1,000	0
TOTAL	5,000	5,010	10

5. Data Quality Engine (Notebook 2)

5.1 Quality Check Logic

The quality engine implements three structured validation checks. Each check scans the entire dataset for a specific defect class and logs every failing record with its broker source, trade identifier, a descriptive issue label, and a severity classification:

Check #	Check Name	Logic	Severity	Issues Found
CHECK 1	Missing Price	Price field is null / NaN	CRITICAL	101
CHECK 2	Price Anomaly	Price > 5,000 or Price <= 0	HIGH	53
CHECK 3	Duplicate Trade ID	TradeID appears more than once	MEDIUM	10

5.2 Output Structure

Upon completion, the engine produces three structured output files:

- `detailed_error_log.csv`: A row-level audit trail. Every single flagged record is listed with its Broker, TradeID, Issue_Type, and Severity — enabling precise investigation of any individual trade.
- `broker_error_summary.csv`: An aggregated metric log showing the total number of issues per broker — providing pre-calculated operational checkpoints that support your executive dashboard views.
- `broker_issue_matrix.csv`: A pivot-style matrix cross-tabulating each broker against each issue category — used to identify which broker is most prone to which defect type.

5.3 Findings: Issue Breakdown by Broker

Broker	Primary Issue	Missing Price	Price Anomaly	Duplicates
Apex Securities	Missing Price	19	15	5
Beacon Trades	Missing Price	21	13	5
Horizon Brokers	Missing Price	23	6	0
Summit Clearing	Price Anomaly	13	10	0
Vanguard Feeds	Missing Price	25	9	0
TOTAL	Missing Price	101	53	10

5.4 Findings: Severity Classification

Severity Level	Issue Type	Count
CRITICAL	Missing Price (Null)	101
HIGH	Price Anomaly (Outlier)	53
MEDIUM	Duplicate Trade ID	10
TOTAL		164

Key insight: 61.6% of all detected issues are classified as CRITICAL (missing prices), meaning more than 6 in 10 flagged records would cause an immediate downstream failure if loaded into

a production system without prior validation. This underscores the operational value of the pipeline.

6. Relational Database Layer (MySQL)

6.1 Schema Design

The MySQL relational layer provides a persistent, queryable store for the error log produced by the quality engine. The schema is intentionally lean — a single `broker_dataops_db` database with one operational table (`feed_errors`) — designed to handle high-volume error record ingestion with minimal overhead.

The `feed_errors` table captures four fields per flagged record:

- `id` (INT AUTO_INCREMENT PRIMARY KEY): A system-generated unique identifier for each error record, enabling precise row-level referencing during investigation and escalation workflows.
- `Broker` (VARCHAR 50): The name of the source broker — used as the primary grouping dimension in all analytical queries and dashboard views.
- `TradeID` (VARCHAR 30): The original trade identifier from the broker feed — enables cross-referencing back to the raw input file for root cause investigation.
- `Issue_Type` (VARCHAR 100): A descriptive label for the defect class (Missing Price, Price Anomaly, Duplicate Trade ID) plus the specific anomalous value where applicable.
- `Severity` (VARCHAR 20): The priority classification (CRITICAL / HIGH / MEDIUM) for operational triage and escalation routing.

6.2 Analytical SQL Queries

Two SQL analytical queries are implemented in the database layer:

- **Record Verification Query:** A `SELECT * FROM feed_errors LIMIT 10` query to confirm successful data load and validate the structure of imported records.
- **Severity Breakdown Query:** A `GROUP BY Severity` aggregation that counts total issues at each priority level — producing the same severity summary rendered in the Power BI dashboard, providing a direct SQL-to-BI verification pathway.

7. Power BI Executive Dashboard

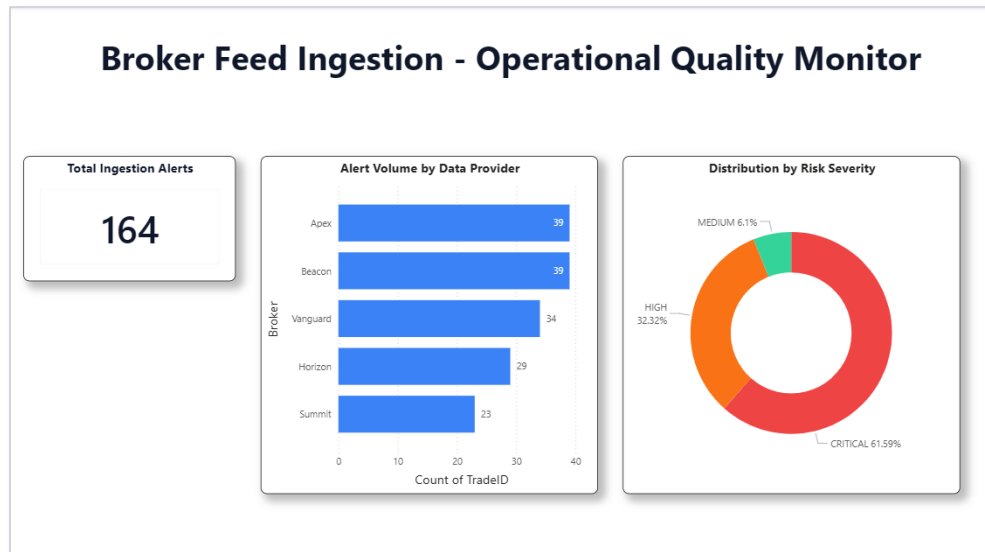


Figure 1.0: Live DataOps Feed Monitor Executive Interface Tracking Broker Data Health Rates.

7.1 Dashboard Purpose

The Power BI dashboard (DataOps_Feed_Monitor.pbix) serves as the operational interface layer of the pipeline — translating raw error log data into an actionable executive view that enables data operations teams to understand feed health, prioritise investigations, and track issue distribution at a glance.

7.2 Data Sources Connected

- **Analytical Reporting Dataset:** Connects to the structured validation metrics to aggregate dynamic record counts, driving the executive KPI scorecards, broker-level error volumes, and risk severity distributions.
- **detailed_error_log.csv:** Populates the drillable issue register table for granular investigation

7.3 Dashboard Components

- **KPI Cards:** At-a-glance metrics showing total records processed, total issues flagged, critical issue count, and broker count — the core health indicators for a DataOps morning review.
- **Broker Health Scorecards:** A visual breakdown of error counts by broker, enabling immediate identification of which feed source requires priority investigation or escalation.
- **Issue Type Distribution:** A categorical breakdown showing the proportion of CRITICAL, HIGH, and MEDIUM issues — helping operations teams understand whether today's feed problems are systemic (high CRITICAL count) or operational (high DUPLICATE count).
- **Broker-Issue Matrix View:** A cross-filter enabled view showing each broker's specific defect profile — enabling targeted root cause investigation and pattern recognition across feed sources.
- **Drillable Issue Register:** A table view of the full detailed error log, filterable by broker, issue type, and severity — the investigation starting point for any flagged trade.

8. Key Findings & Recommendations

8.1 Critical Findings

- Missing prices (null values) represent the single largest defect category at 101 occurrences (61.6% of total issues) — a systemic problem distributed across all five broker feeds with no single outlier broker responsible.
- Price anomalies (extreme values) account for 53 issues (\$32.3\%\$) and are particularly concentrated in Summit Clearing (**10** high-price anomalies) and Apex Securities (**15** negative-price records) — suggesting potential upstream system-specific encoding failures at these endpoints.
- Duplicate Trade IDs, while the smallest category at 10 records (\$6.1\%\$), are exclusively concentrated in two brokers (Apex and Beacon, with 5 records each) — indicating a double-submission pattern at those specific trading gateways that warrants broker-side investigation.

8.2 Operational Recommendations

- **Immediate:** Implement pre-load validation gates in the production ingestion pipeline — no record with a null price field should be permitted to reach the database. Route CRITICAL records to a quarantine table pending manual review or broker resubmission.
- **Short-term:** Engage Apex Securities and Beacon Trades on the duplicate submission pattern. Request audit logs from their trade gateway systems to identify whether the duplication is occurring at submission or during file generation.

- Medium-term: Establish daily automated pipeline execution with threshold alerting — if any broker's CRITICAL issue rate exceeds 3% of their daily feed volume, trigger an automatic escalation notification to the Data Analyst Manager.
- Long-term: Extend the quality engine with additional checks: timestamp validation (trades outside market hours), symbol validity (against a reference security master), and quantity range checks — building toward a comprehensive pre-load data quality framework.

9. Skills & Competencies Demonstrated

9.1 Technical Skills

- Python Data Engineering: End-to-end pipeline design using Pandas and NumPy for large dataset ingestion, cleaning, outlier detection, and structured output generation.
- SQL & Relational Database Design: MySQL schema creation, table normalisation, data import workflows, and analytical GROUP BY queries for operational metrics.
- ETL Pipeline Concepts: Designed a complete Extract-Transform-Load pipeline from raw CSV ingestion through transformation, quality validation, and loading to both a relational store and a BI tool.
- Power BI Dashboard Development: Multi-source data model connecting validation metrics layers; built interactive KPI cards, cross-filter slicers, matrix visuals, and a drillable data table for end-to-end operational use.
- Data Quality Methodology: Implemented industry-standard data quality check categories — completeness (null checks), validity (range/constraint checks), and uniqueness (duplicate checks) — with severity classification.

9.2 Operational & Analytical Skills

- Root Cause Documentation: Every flagged issue includes broker source, trade identifier, defect description, and severity — enabling structured root cause investigation and escalation.
- Stakeholder Communication: Dashboard design prioritises operational readability — non-technical stakeholders can immediately understand feed health and required actions without interpreting raw data.
- Issue Triage & Escalation: Three-tier severity classification (CRITICAL / HIGH / MEDIUM) provides a clear operational escalation framework aligned with DataOps best practices.

- Analytical Problem-Solving: Identified systemic vs. broker-specific patterns in the findings (e.g., null prices are cross-broker; duplicate IDs are broker-isolated), demonstrating the ability to draw actionable insights from structured error data.

10. Conclusion

The Broker Feed Data Quality Monitor successfully demonstrates an automated, end-to-end DataOps pipeline capable of ingesting, validating, and reporting on financial trade data feeds from multiple broker sources. By combining Python-based quality engineering with MySQL relational logging and a Power BI executive interface, the project delivers a production-ready framework that directly addresses the real-world data reliability challenges faced by compliance-focused financial platforms.

Across 5,010 simulated trade records from five broker endpoints, the pipeline identified 164 data quality issues - including 101 CRITICAL missing-price records that would have caused downstream system failures if loaded without validation. Each issue is precisely documented with broker attribution, trade-level granularity, and severity classification, enabling immediate operational response.

The project architecture reflects the layered design principles of modern DataOps practice: separation of ingestion, transformation, and presentation layers; relational persistence for audit and query; and an executive dashboard that translates technical findings into operational decisions. These are the same principles that underpin enterprise-grade compliance data platforms serving regulated financial institutions.

This project is part of Amegha V.S 's professional portfolio demonstrating readiness for a Data Analyst role. All data used in this project is fully synthetic and generated for demonstration purposes only.
